

Caderno de Exercícios Resolvidos

Introdução à Teoria da Computação (ACH2043) • Linguagens Livres-do-Contexto

Este caderno reúne **todos os exercícios** das sete aulas sobre linguagens livres-do-contexto, organizados por assunto. Cada exercício traz uma **etiqueta do tópico**, o enunciado e uma **resolução comentada** — o objetivo não é só dar a resposta, mas mostrar o *raciocínio* que leva até ela, para você conseguir reproduzir a técnica em variações do problema na prova.

Como usar este caderno

- Cubra a parte da «Resolução» e tente resolver primeiro; depois compare.
- Preste atenção às **etiquetas de assunto** — elas indicam qual técnica está sendo treinada (projetar GLC, converter para FNC, rodar o CYK, projetar AP, etc.).
- As respostas de "gramática/autômato" quase nunca são únicas: se a sua for diferente, teste-a gerando/reconhecendo algumas cadeias para conferir.

1. Derivação e leitura de gramáticas

Estes exercícios treinam o mecanismo básico de uma GLC: partir da variável inicial e aplicar regras até chegar a uma cadeia de terminais. Saber **derivar** uma cadeia (e reconhecer a linguagem gerada) é pré-requisito para tudo o que vem depois.

GLC · DERIVAÇÃO

1.1 — Derivar a cadeia 000#111 na gramática G1

Enunciado.

Dada G1 com as regras abaixo, mostre uma derivação de 000#111.

G1

$$A \rightarrow 0A1$$
$$A \rightarrow B$$
$$B \rightarrow \#$$

Resolução.

A ideia é aplicar $A \rightarrow 0A1$ tantas vezes quantas forem os zeros desejados (cada aplicação adiciona um 0 à esquerda e um 1 à direita, mantendo A no meio), e só então "fechar" com $A \rightarrow B \rightarrow \#$:

$$A \Rightarrow 0A1 \Rightarrow 00A11 \Rightarrow 000A111 \Rightarrow 000B111 \Rightarrow 000\#111$$

Passo a passo: as três primeiras setas usam $A \rightarrow 0A1$ (criando o bloco 000...111); a quarta usa $A \rightarrow B$ (o A central vira B); a quinta usa $B \rightarrow \#$ (o B vira o símbolo central). Como não há mais variáveis, a derivação termina. A cadeia tem $n = 3$ zeros e três uns, com o # no meio, confirmando o padrão de $L(G1) = \{ 0^n \# 1^n \}$. ✓

1.2 — A cadeia (1(0)) pertence a L(G3)?

Enunciado.

Dada G3 com a regra $S \rightarrow (S) \mid SS \mid 0 \mid 1$, decida se $(1(0)) \in L(G3)$, exibindo uma derivação caso pertença.

Resolução.

Para provar que uma cadeia pertence à linguagem, basta exibir **uma** derivação que a gere.

Analisando a cadeia (1(0)): ela é um par de parênteses externos envolvendo «1(0)», que por sua vez é a justaposição de «1» e «(0)». Isso sugere usar $S \rightarrow (S)$, depois $S \rightarrow SS$, e resolver cada parte:

$$S \Rightarrow (S) \Rightarrow (SS) \Rightarrow (1S) \Rightarrow (1(S)) \Rightarrow (1(0))$$

Detalhe de cada passo: $S \rightarrow (S)$ cria os parênteses externos; $S \rightarrow SS$ divide o interior em duas partes; a primeira parte vira 1 ($S \rightarrow 1$); a segunda vira (S) e depois (0). Como conseguimos uma derivação completa, $(1(0)) \in L(G3)$. ✓ Observação: essa gramática gera cadeias binárias com parênteses corretamente aninhados.

1.3 — Derivar $a + a \times a$ em G4 respeitando a precedência

Enunciado.

Dada a gramática de expressões G4 abaixo, derive $a + a \times a$ de modo que a estrutura reflita a precedência de $\times$ sobre $+$.

G4

$$\begin{aligned} \langle \text{EXPR} \rangle &\rightarrow \langle \text{EXPR} \rangle + \langle \text{TERMO} \rangle \mid \langle \text{TERMO} \rangle \\ \langle \text{TERMO} \rangle &\rightarrow \langle \text{TERMO} \rangle \times \langle \text{FATOR} \rangle \mid \langle \text{FATOR} \rangle \\ \langle \text{FATOR} \rangle &\rightarrow (\langle \text{EXPR} \rangle) \mid a \end{aligned}$$

Resolução.

A precedência é codificada pela **hierarquia das variáveis**: a soma vive no nível de $\langle \text{EXPR} \rangle$ (mais "externo") e o produto no nível de $\langle \text{TERMO} \rangle$ (mais "interno"). Como o $\times$ está mais fundo na árvore, ele é avaliado primeiro. Derivando (usamos E, T, F para abreviar):

$$\begin{aligned} E &\Rightarrow E + T \Rightarrow T + T \Rightarrow F + T \Rightarrow a + T \\ &\Rightarrow a + T \times F \Rightarrow a + F \times F \Rightarrow a + a \times F \Rightarrow a + a \times a \end{aligned}$$

O ponto-chave é o passo $E \Rightarrow E + T$: ele separa a soma em «E» (que vira a) e «T» (que vira $a \times a$). Como $a \times a$ nasce agrupado dentro de um único $\langle \text{TERMO} \rangle$, a multiplicação fica "presa" como uma unidade — exatamente o comportamento de precedência esperado. Se $a = 3$, a expressão vale $3 + 9 = 12$ (e não 18). ✓

1.4 — Qual é a linguagem de $S \rightarrow aSb \mid \varepsilon$?

Enunciado.

Descreva $L(G)$ para a gramática $G: S \rightarrow aSb \mid \varepsilon$.

Resolução.

A estratégia é gerar as primeiras cadeias e procurar o padrão. Cada aplicação de $S \rightarrow aSb$ envolve o S restante entre um «a» (à esquerda) e um «b» (à direita); a regra $S \rightarrow \varepsilon$ encerra:

$$\begin{aligned} S &\Rightarrow \varepsilon \\ S &\Rightarrow aSb \Rightarrow ab \\ S &\Rightarrow aSb \Rightarrow aaSbb \Rightarrow aabb \end{aligned}$$

Em cada nível a regra recursiva acrescenta **simultaneamente** um a na frente e um b no fim, então o número de a's é sempre igual ao de b's, e todos os a's vêm antes de todos os b's. Concluimos:

$$L(G) = \{ a^n b^n \mid n \geq 0 \}$$

Esta é a linguagem não-regular clássica — a recursão "casada" da regra aSb faz o papel da contagem que um autômato finito não consegue realizar. ✓

2. Projeto de gramáticas livres-do-contexto

Aqui o desafio é o inverso: dada uma linguagem, **construir** uma GLC que a gere. As técnicas recorrentes são: união ($S \rightarrow S1 \mid S2$), concatenação ($S \rightarrow S1 S2$), regras recursivas "casadas" ($R \rightarrow uRv$) para partes que precisam corresponder, e conversão a partir de autômatos/expressões regulares.

2.1 — GLC para cadeias que começam e terminam com «a»

Enunciado.

Construa uma GLC para $L = \{ \text{cadeias sobre } \{a,b\} \text{ que começam e terminam com } a \}$. A expressão regular é $a\Sigma^*a \cup \{a\}$.

Resolução.

A parte $a\Sigma^*a$ é um **aninhamento**: fixamos um «a» no início e um «a» no fim, e no meio colocamos uma variável auxiliar T que gera qualquer coisa (Σ^*). Depois somamos a cadeia unitária «a» (que também começa e termina com a):

$$\begin{aligned} S &\rightarrow a T a \mid a \\ T &\rightarrow Ta \mid Tb \mid \varepsilon \end{aligned}$$

Verificação: $S \rightarrow a$ gera «a» ✓; $S \rightarrow aTa \rightarrow aa$ (com $T \rightarrow \varepsilon$) gera «aa» ✓; $S \rightarrow aTa \rightarrow aba$ (com $T \rightarrow Tb \rightarrow b$) gera «aba» ✓. A variável T , sozinha, gera todas as cadeias sobre $\{a,b\}$ (inclusive a vazia), garantindo o «qualquer coisa no meio». ✓

2.2 — GLC para cadeias que começam e terminam com o mesmo símbolo

Enunciado.

Construa uma GLC para $L = \{ \text{cadeias sobre } \{a,b\} \text{ que começam e terminam com o mesmo símbolo} \}$.

Resolução.

Perceba que L é a **união** de duas linguagens: $L_1 = \{ \text{começa e termina com } a \}$ e $L_2 = \{ \text{começa e termina com } b \}$. A técnica da união é criar uma variável inicial que escolhe entre as duas sub-gramáticas:

$$\begin{aligned} S &\rightarrow S1 \mid S2 \\ S1 &\rightarrow a \mid a T a \\ S2 &\rightarrow b \mid b T b \\ T &\rightarrow Ta \mid Tb \mid \varepsilon \end{aligned}$$

$S1$ reaproveita a solução do exercício anterior (para «a»), e $S2$ é a versão simétrica para «b». A regra $S \rightarrow S1 \mid S2$ combina as duas. Reutilizamos a mesma variável T (gerador de Σ^*) em ambas — não há problema em compartilhá-la. ✓

2.3 — GLC para $L = \{ a^m b^m c^n d^n \mid m, n \geq 0 \}$

Enunciado.

Construa uma GLC para a linguagem em que um bloco $a^m b^m$ é seguido de um bloco $c^n d^n$ (m e n independentes).

Resolução.

A linguagem é a **concatenação** $L_1 \cdot L_2$, com $L_1 = \{ a^m b^m \}$ e $L_2 = \{ c^n d^n \}$. Já sabemos gerar cada bloco com a técnica recursiva casada (como em $a^n b^n$). Para concatenar, a variável inicial simplesmente gera uma sub-gramática seguida da outra:

$$\begin{aligned} S &\rightarrow S1 S2 \\ S1 &\rightarrow a S1 b \mid \varepsilon \\ S2 &\rightarrow c S2 d \mid \varepsilon \end{aligned}$$

Como $S1$ e $S2$ são independentes, m e n podem ser diferentes. Ex.: $S \Rightarrow S1 S2 \Rightarrow a S1 b S2 \Rightarrow ab S2 \Rightarrow ab c S2 d \Rightarrow ab cd = abcd$ ($m=1, n=1$). ✓

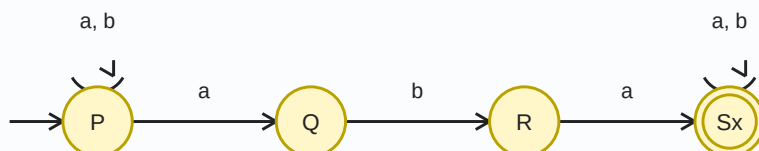
2.4 — GLC para $\{ w \in \{a,b\}^* \mid w \text{ contém a subcadeia aba} \}$

Enunciado.

Construa uma GLC para as cadeias que contêm «aba» em algum lugar. (Sugestão: projete antes o autômato.)

Resolução.

Caminho 1 (autômato → GLC). Primeiro um AFN: um estado P que consome livremente a's e b's, avança por $a \rightarrow b \rightarrow a$ até um estado de aceitação S que também consome o resto:



Convertendo cada estado numa variável (loops e transições viram regras; o estado de aceitação ganha $\rightarrow \epsilon$):

$$P \rightarrow aP \mid bP \mid aQ$$

$$Q \rightarrow bR$$

$$R \rightarrow aS$$

$$S \rightarrow aS \mid bS \mid \epsilon$$

Caminho 2 (expressão regular → GLC). Da ER $\Sigma^*aba\Sigma^*$ sai uma versão mais enxuta para a mesma linguagem — geralmente é o que se prefere escrever:

$$S \rightarrow T a b a T$$

$$T \rightarrow aT \mid bT \mid \epsilon$$

T gera qualquer prefixo/sufixo, e o miolo «aba» é forçado no meio. ✓

Exercício 2.4 do livro — GLCs sobre $\Sigma = \{0, 1\}$

Nos itens abaixo, o objetivo é projetar uma GLC para cada linguagem descrita.

2.5(a) — { w | w contém pelo menos três 1s }

Enunciado.

Cadeias binárias com três ou mais 1s.

Resolução.

Forçamos **três 1s obrigatórios** e permitimos "lixo" arbitrário ($X = \Sigma^*$) antes, entre e depois deles:

$$S \rightarrow X 1 X 1 X 1 X$$

$$X \rightarrow 0X \mid 1X \mid \varepsilon$$

Os três «1» literais na regra de S garantem o mínimo de três uns; os quatro X permitem qualquer quantidade de símbolos (0 ou 1) em volta. Assim cadeias como «111», «01011001» (que tem ≥ 3 uns) são geradas. ✓

2.5(b) — { w | w começa e termina com o mesmo símbolo }

Enunciado.

Cadeias binárias que começam e terminam com o mesmo símbolo.

Resolução.

Mesma técnica de união do exercício 2.2, adaptada a $\{0,1\}$. Incluímos as cadeias unitárias 0 e 1 (que trivialmente começam e terminam com o mesmo símbolo):

$$S \rightarrow 0 T 0 \mid 1 T 1 \mid 0 \mid 1$$

$$T \rightarrow 0T \mid 1T \mid \varepsilon$$

A regra $0T0$ fixa 0 nas pontas com qualquer coisa no meio; $1T1$ faz o simétrico; 0 e 1 cobrem os casos de comprimento 1. ✓

2.5(c) — { w | o comprimento de w é ímpar }

Enunciado.

Cadeias binárias de comprimento ímpar.

Resolução.

Ideia: um símbolo central obrigatório (comprimento 1), e a possibilidade de acrescentar **dois** símbolos por vez, o que preserva a paridade ímpar:

$$S \rightarrow 0 \mid 1 \mid 00S \mid 01S \mid 10S \mid 11S$$

Cada regra com dois símbolos à frente soma 2 ao comprimento (ímpar continua ímpar); os casos base 0 e 1 têm comprimento 1. Ex.: 01011 (comprimento 5) sai de $S \Rightarrow 01S \Rightarrow 0101S$? — na verdade $S \Rightarrow 01S \Rightarrow 0101S \Rightarrow 01011$. ✓

2.5(d) — { w | comprimento ímpar e símbolo central é 0 }

Enunciado.

Cadeias binárias de comprimento ímpar cujo símbolo do meio é 0.

Resolução.

Construímos a cadeia **de fora para dentro**, acrescentando um símbolo (qualquer) em cada ponta a cada passo, até chegar ao centro — que é forçado a ser 0:

$$S \rightarrow 0 S 0 \mid 0 S 1 \mid 1 S 0 \mid 1 S 1 \mid 0$$

A regra recursiva adiciona um símbolo à esquerda e outro à direita (mantendo o centro no meio); o caso base $S \rightarrow 0$ planta o símbolo central obrigatório. Ex.: 11010 — não, comprimento par; correto: 1 1 0 0 1 sai de $S \Rightarrow 1S1 \Rightarrow 1 0 1$ (centro 0). ✓

2.5(e) — { w | w = w^R (palíndromo) }

Enunciado.

Cadeias binárias que são palíndromos (iguais ao seu reverso).

Resolução.

Um palíndromo se constrói de fora para dentro colocando o **mesmo** símbolo nas duas pontas. Os casos base cobrem comprimento par (ϵ) e ímpar (0 ou 1 no centro):

$$S \rightarrow 0 S 0 \mid 1 S 1 \mid 0 \mid 1 \mid \epsilon$$

As regras 0S0 e 1S1 garantem simetria (mesmo símbolo em posições espelhadas); ϵ dá os palíndromos de comprimento par no centro, e 0/1 dão os de comprimento ímpar. Ex.: 10101 sai de $S \Rightarrow 1S1 \Rightarrow 1 0S0 1 \Rightarrow 1 0 1 0 1$. ✓

2.5(f) — O conjunto vazio $\emptyset$

Enunciado.

Construa uma GLC que gere a linguagem vazia.

Resolução.

A linguagem vazia não contém **nenhuma** cadeia — nem a vazia. Basta uma gramática cuja variável inicial nunca consiga terminar uma derivação (nunca chega a uma cadeia só de terminais):

$$S \rightarrow S$$

Como o único jeito de reescrever S é por outro S, jamais eliminamos a variável, logo nenhuma cadeia de terminais é produzida: $L(G) = \emptyset$. ✓ (Atenção: isto é diferente de $S \rightarrow \epsilon$, que geraria a linguagem $\{\epsilon\}$, contendo a cadeia vazia.)

2.6 — Adapte $S \rightarrow aSb \mid \epsilon$ para $L = \{ a^n b^{2n} \mid n \geq 0 \}$

Enunciado.

Modifique a gramática de $a^n b^n$ para que cada a corresponda a dois b's.

Resolução.

Na regra casada $R \rightarrow uRv$, cada aplicação insere «u» à esquerda e «v» à direita. Para obter **dois** b por a, basta que «v» sejam dois b's:

$$S \rightarrow a S b b \mid \epsilon$$

Derivando: $S \Rightarrow aSbb \Rightarrow aaSbbbb \Rightarrow aabbbb$, ou seja $a^2 b^4$ ($n = 2$). Em cada nível entra 1 «a» e 2 «b», mantendo a razão 1:2 entre a's e b's. ✓

2.7 — GLC para $\{ a^i b^j c^k \mid i=j \text{ ou } j=k \}$

Enunciado.

Construa uma GLC para a linguagem em que o número de a's iguala o de b's, OU o de b's iguala o de c's. (Esta linguagem é inerentemente ambígua.)

Resolução.

Vemos a linguagem como a **união** de dois casos: $L1 = \{ a^i b^i c^* \}$ (casa a com b, c livre) e $L2 = \{ a^* b^j c^j \}$ (casa b com c, a livre). Construímos uma sub-gramática para cada e unimos:

$$S \rightarrow L1 \mid L2$$

$$L1 \rightarrow X C$$

$$X \rightarrow a X b \mid \epsilon$$

$$C \rightarrow c C \mid \epsilon$$

$$L2 \rightarrow A Y$$

$$A \rightarrow a A \mid \epsilon$$

$$Y \rightarrow b Y c \mid \epsilon$$

Em L1, X gera $a^i b^i$ (a casado com b) e C gera c's livres; em L2, A gera a's livres e Y gera $b^j c^j$ (b casado com c). A cadeia aaabbbccc pode ser gerada por **ambos** os ramos (por L1 com $i=j=3$ e $C=ccc$, ou por L2 com $j=k=3$ e $A=aaa$) — é esse duplo caminho que caracteriza a ambiguidade **inerente** desta linguagem. ✓

2.8 — Mostre que a frase é ambígua em G2

Enunciado.

Mostre que «the girl touches the boy with the flower» tem duas interpretações na gramática G2 do inglês.

Resolução.

A ambiguidade está em **onde o «with the flower» se conecta**. Há duas árvores sintáticas possíveis, correspondendo a dois significados:

- **Leitura 1 (a flor é o instrumento):** a garota, *usando a flor*, toca o menino. O sintagma preposicional «with the flower» liga-se ao <VERB-PHRASE> (ao verbo «touches»).
- **Leitura 2 (a flor está com o menino):** a garota toca o menino *que está com a flor*. O «with the flower» liga-se ao <NOUN-PHRASE> «the boy».

Como a mesma cadeia admite duas árvores sintáticas estruturalmente distintas (o <PREP-PHRASE> pendura em pontos diferentes), G2 é **ambígua**. Este é o análogo linguístico do problema de precedência que motivou G4 vs. G5. ✓

3. Forma Normal de Chomsky

A FNC padroniza toda regra para a forma $V \rightarrow WX$ (duas variáveis) ou $V \rightarrow a$ (um terminal). Estes exercícios treinam a **conversão** e o entendimento das propriedades que tornam a FNC útil (árvores binárias, derivações de tamanho fixo $2n-1$, base do CYK).

3.1 — Converta $S \rightarrow aSb \mid \epsilon$ para a Forma Normal de Chomsky

Enunciado.

Aplique os quatro passos de conversão à gramática $S \rightarrow aSb \mid \epsilon$.

Resolução.

Passo 1 — nova variável inicial (garante que a inicial não apareça à direita):

$$\begin{aligned} S_0 &\rightarrow S \\ S &\rightarrow aSb \mid \epsilon \end{aligned}$$

Passo 2 — eliminar regras- ϵ . Removemos $S \rightarrow \epsilon$ e compensamos onde S aparecia: em «aSb» (surge «ab») e em « $S_0 \rightarrow S$ » (surge $S_0 \rightarrow \epsilon$):

$$\begin{aligned} S_0 &\rightarrow S \mid \epsilon \\ S &\rightarrow aSb \mid ab \end{aligned}$$

Passo 3 — eliminar regras unitárias. Removemos $S_0 \rightarrow S$ copiando as produções de S para S_0 :

$$\begin{aligned} S_0 &\rightarrow aSb \mid ab \mid \epsilon \\ S &\rightarrow aSb \mid ab \end{aligned}$$

Passo 4 — padronizar. Criamos $A \rightarrow a$ e $B \rightarrow b$ para isolar os terminais, e quebramos as regras de 3 símbolos com uma auxiliar $X = \langle Sb \rangle$:

$$\begin{aligned} S_0 &\rightarrow A X \mid A B \mid \epsilon \\ S &\rightarrow A X \mid A B \\ X &\rightarrow S B \\ A &\rightarrow a \quad B \rightarrow b \end{aligned}$$

Agora toda regra é $V \rightarrow WX$ ou $V \rightarrow a$, e só S_0 tem a regra $\rightarrow \epsilon$. A gramática está em FNC e gera a mesma linguagem $\{a^n b^n\}$. ✓

3.2 — Por que a FNC é útil para algoritmos?

Enunciado.

Explique por que colocar uma GLC na FNC ajuda a construir algoritmos de análise sintática.

Resolução.

Dois fatos, juntos, viabilizam algoritmos eficientes:

- **Árvores binárias e tamanho previsível:** na FNC toda regra ou junta duas variáveis ($V \rightarrow WX$) ou emite um terminal ($V \rightarrow a$). Logo a árvore sintática é binária, e a derivação de uma cadeia de tamanho n tem **exatamente $2n - 1$ passos** ($n - 1$ do tipo WX e n do tipo terminal).
- **Decomposição recursiva:** uma variável gera uma subcadeia **se e somente se** essa subcadeia puder ser quebrada em duas metades geradas por variáveis W e X , com uma regra $V \rightarrow WX$.

Essa segunda propriedade permite preencher uma tabela de subcadeias de baixo para cima (bottom-up) e decidir a pertinência em tempo polinomial $O(n^3)$ — é exatamente o que o algoritmo CYK faz. Sem a FNC, teríamos de testar derivações de tamanhos variados, sem essa estrutura regular. ✓

3.3 — Quantos passos tem a derivação de «aabb» numa GLC na FNC?

Enunciado.

Use a propriedade do número de passos para responder sem derivar explicitamente.

Resolução.

A cadeia «aabb» tem comprimento $n = 4$. Pela propriedade da FNC, a derivação de uma cadeia de tamanho n tem **exatamente $2n - 1$ passos**:

$$2n - 1 = 2 \times 4 - 1 = 7 \text{ passos}$$

Desses 7, são $n - 1 = 3$ aplicações de regras $V \rightarrow WX$ (que constroem a árvore binária de 4 folhas) e $n = 4$ aplicações de $V \rightarrow a$ (que colocam cada terminal nas folhas). A conta $3 + 4 = 7$ confere. ✓

4. Algoritmo CYK

O CYK decide se uma cadeia w pertence a $L(G)$, dada G na FNC, preenchendo uma matriz triangular de baixo para cima. Estes exercícios treinam a execução do algoritmo e a análise de seu custo. A gramática de referência (para a linguagem de cadeias com igual número de a 's e b 's) é:

Gramática de referência (FNC)

$S \rightarrow AT \mid BU \mid SS \mid AB \mid BA \mid \varepsilon$

$S \rightarrow AT \mid BU \mid SS \mid AB \mid BA$

$T \rightarrow SB \quad U \rightarrow SA \quad A \rightarrow a \quad B \rightarrow b$

4.1 — Rode o CYK para a cadeia «ab»

Enunciado.

Determine, pelo CYK, se «ab» pertence à linguagem (igual número de a's e b's).

Resolução.

A cadeia tem $n = 2$. **Passo 1 (diagonal):** para cada posição, colocamos a variável que gera aquele terminal. $w_1 = a \rightarrow M(1,1) = \{A\}$ (regra $A \rightarrow a$); $w_2 = b \rightarrow M(2,2) = \{B\}$.

Passo tamanho 2: para $M(1,2)$ há uma única quebra ($k = 1$). Combinamos $W \in M(1,1) = \{A\}$ com $X \in M(2,2) = \{B\}$ e procuramos regras $V \rightarrow AB$. Existem: $S \rightarrow AB$ e $S0 \rightarrow AB$. Logo $M(1,2) = \{S0, S\}$.

	1	2
	a	b
1	A	S0, S
2		B

Como $S0 \in M(1,2)$, a cadeia «ab» é **ACEITA** — de fato ela tem um a e um b, quantidades iguais. ✓

4.2 — Rode o CYK para a cadeia «aab»

Enunciado.

Determine, pelo CYK, se «aab» pertence à linguagem.

Resolução.

$n = 3$. **Diagonal:** $M(1,1) = \{A\}$, $M(2,2) = \{A\}$, $M(3,3) = \{B\}$.

Tamanho 2: $M(1,2) = \{\text{«aa»}\}$ precisaria de uma regra $V \rightarrow AA$, que não existe $\rightarrow \emptyset$. $M(2,3) = \{\text{«ab»}\}$ tem $V \rightarrow AB \rightarrow \{S0, S\}$.

Tamanho 3: $M(1,3)$ testa duas quebras. Em $k=1$: A (de $M(1,1)$) com $\{S0, S\}$ (de $M(2,3)$) — não há regra $V \rightarrow A \cdot S0$ nem $V \rightarrow A \cdot S$. Em $k=2$: $M(1,2) = \emptyset$ com B — uma parte é vazia. Logo $M(1,3) = \emptyset$.

	1	2	3
	a	a	b
1	A	$\emptyset$	$\emptyset$
2		A	S0, S
3			B

Como $S0 \notin M(1,3)$, a cadeia «aab» é **REJEITADA** — de fato ela tem dois a's e um b, quantidades diferentes. ✓

4.3 — Qual a complexidade de decidir $w \in L(G)$ pelo CYK?

Enunciado.

Analise o custo de tempo do algoritmo em função de n (tamanho da cadeia) e r (número de regras).

Resolução.

O algoritmo tem três laços aninhados no preenchimento das subcadeias de tamanho ≥ 2 : o tamanho da subcadeia, a posição inicial e o ponto de quebra — cada um variando $O(n)$ vezes, o que dá $O(n^3)$ células/combinações. Em cada uma, testar as regras custa $O(r)$. Portanto:

$$\text{custo} = O(r \cdot n^3) \quad (\text{ou } O(n^3) \text{ se } r \text{ é constante})$$

Isso é **polinomial** — a grande vantagem sobre a força bruta, que testaria todas as derivações de $2n - 1$ passos (custo exponencial). É por isso que a FNC + programação dinâmica são tão importantes. ✓

5. Autômatos com pilha

Um AP reconhece LLCs usando uma pilha como memória auxiliar. Estes exercícios treinam o **projeto** de APs (a descrição informal da estratégia costuma valer tanto quanto o diagrama) e a **simulação** de computações. Notação das transições: $\langle a, b ; c \rangle =$ lê a da fita, desempilha b , empilha c ($\lambda =$ nada).

5.1 — AP para $\{ w \in \{0,1\}^* \mid w \text{ tem pelo menos tantos 0's quanto 1's} \}$

Enunciado.

Projete um AP (descrição informal) para a linguagem em que $\#0 \geq \#1$.

Resolução.

Estratégia: usar a pilha para registrar o "saldo" entre 0's e 1's. Empilhamos $\$$ como sentinela; para cada 0 lido, empilhamos um marcador X; para cada 1 lido, se há um X no topo, desempilhamos (esse 1 casou com um 0 anterior); se não há X (só o $\$$), empilhamos um Y (registra um 1 sem par).

Condição de aceitação: ao fim da entrada, aceite se **não houver nenhum Y** na pilha — ou seja, todo 1 encontrou um 0 correspondente, garantindo $\#0 \geq \#1$. Os X que eventualmente sobrem representam 0's em excesso, o que é permitido. ✓

5.2 — AP para $\{0^i 1^j \mid j \geq i\}$

Enunciado.

Projete um AP para as cadeias com um bloco de 0's seguido de um bloco de 1's, com pelo menos tantos 1's quanto 0's.

Resolução.

Empilhamos um marcador por 0; ao ler os 1's, desempilhamos um marcador por 1 (casando cada 1 com um 0). Quando a pilha esvazia (todos os i zeros casados), os 1's extras são lidos livremente — é isso que permite $j > i$:

```
q0 --(λ,λ;$)--> q1      (empilha sentinela)
q1:  0, λ ; 0           (empilha um 0 por leitura)
q1 --(λ,λ;λ)--> q2      (acaba a fase de 0s)
q2:  1, 0 ; λ           (casa um 1 com cada 0)
q2 --(λ,$;λ)--> q3      (esvaziou a pilha)
q3:  1, λ ; λ           (1's extras: j > i)
```

O estado de aceitação $q3$ permite os 1's excedentes; como só chegamos a $q3$ depois de casar todos os 0's, garantimos $j \geq i$. (Se faltasse 0 para um 1, o ramo falharia.) ✓

5.3 — AP para $\{0^n 1^{2n} \mid n \geq 0\}$

Enunciado.

Projete um AP em que cada 0 corresponda a exatamente dois 1's.

Resolução.

A dica é empilhar **dois** marcadores por 0 lido, e depois desempilhar um por 1. Assim a pilha só esvazia quando o número de 1's for o dobro do de 0's:

```
q0 --(λ,λ;$)--> q1
q1:  0, λ ; 00          (empilha DOIS zeros por 0 lido)
q1 --(λ,λ;λ)--> q2
q2:  1, 0 ; λ           (desempilha um zero por 1 lido)
q2 --(λ,$;λ)--> q3      (aceita)
```

Ex.: para «001111» ($n=2$), empilhamos 4 zeros e desempilhamos 4 (um por 1) — a pilha esvazia exatamente ao fim. Cadeias como «01» seriam rejeitadas (sobraria um zero). ✓

5.4 — Simule o AP de $0^n 1^n$ sobre a cadeia 0011

Enunciado.

Mostre a computação do AP (empilha 0's, casa com 1's) passo a passo sobre «0011».

Resolução.

Acompanhamos o estado, a entrada ainda não lida, e o conteúdo da pilha (topo à esquerda):

estado	entrada restante	pilha	
q1	0011	(vazia)	
q2	0011	\$	(empilha sentinela)
q2	011	0\$	(lê 0, empilha 0)
q2	11	00\$	(lê 0, empilha 0)
q3	1	0\$	(lê 1, desempilha 0)
q3	(vazia)	\$	(lê 1, desempilha 0)
q4	(vazia)	(vazia)	(remove \$) → ACEITA

A entrada acaba exatamente quando a pilha esvazia (o sentinela é removido no último passo) → cadeia **aceita**. Se sobrasse símbolo na pilha ou na entrada, seria rejeitada. ✓

6. Equivalência entre AP e GLC

Estes exercícios consolidam a prova de que APs e GLCs descrevem a mesma classe de linguagens. Envolvem a conversão nos dois sentidos e o entendimento das ideias-chave (pilha guarda a derivação; variáveis A_{pq} descrevem trajetos de pilha-vazia a pilha-vazia).

6.1 — Converta $G: S \rightarrow 0S1 \mid \epsilon$ em um autômato com pilha

Enunciado.

Aplique a construção do Lema 2.21 para obter um AP equivalente a G .

Resolução.

A construção usa três estados (início, laço, aceita). O AP empilha o inicial S sobre o sentinela $\$$ e, no laço, para cada variável no topo aplica uma regra (empilhando o lado direito em ordem reversa), e para cada terminal no topo casa com a fita:

```
q_início --( $\lambda, \lambda ; S\$$ )--> q_laço
```

No laço $q_{\text{laço}}$:

```
 $\lambda, S ; 0S1$       (regra  $S \rightarrow 0S1$ , empilhada em ordem  
reversa:  $0$  no topo)
```

```
 $\lambda, S ; \lambda$       (regra  $S \rightarrow \epsilon$ )
```

```
 $0, 0 ; \lambda$       (casa o terminal  $0$ )
```

```
 $1, 1 ; \lambda$       (casa o terminal  $1$ )
```

```
q_laço --( $\lambda, \$ ; \lambda$ )--> q_aceita
```

A regra $S \rightarrow 0S1$ (três símbolos) é empilhada com transições intermediárias, deixando o 0 no topo, depois S , depois 1 no fundo — assim a leitura processa primeiro o 0 , depois expande S , e por fim casa o 1 . ✓

6.2 — Por que empilhar o lado direito das regras em ordem reversa?

Enunciado.

Explique a necessidade de empilhar «em ordem reversa» na conversão $GLC \rightarrow AP$.

Resolução.

Porque a pilha é **LIFO**: o último símbolo empilhado é o primeiro a ser retirado (fica no topo). Ao aplicar uma regra $A \rightarrow xyz$, queremos que a derivação continue processando x , depois y , depois z (a ordem natural de leitura da cadeia). Para isso, x precisa estar no topo — então empilhamos na ordem inversa: primeiro z (vai ao fundo), depois y , e por último x (fica no topo).

Se empilhássemos na ordem direta (x , depois y , depois z), o topo seria z e processaríamos a cadeia invertida (zyx), gerando a linguagem errada. A inversão corrige isso. ✓

6.3 — Simule o AP de $S \rightarrow 0S1 \mid \varepsilon$ sobre a entrada «0011»

Enunciado.

Mostre como a pilha do AP reproduz a derivação de «0011».

Resolução.

Em G, a derivação é $S \Rightarrow 0S1 \Rightarrow 00S11 \Rightarrow 0011$. A pilha do AP (topo à esquerda) acompanha, alternando entre expandir a variável do topo e casar terminais:

pilha (topo→fundo)	ação
S \$	empilha inicial
0 S 1 \$	aplica $S \rightarrow 0S1$
S 1 \$	lê 0, casa com o topo
0 S 1 1 \$	aplica $S \rightarrow 0S1$
S 1 1 \$	lê 0, casa com o topo
1 1 \$	aplica $S \rightarrow \varepsilon$
1 \$	lê 1, casa com o topo
\$	lê 1, casa com o topo
(vazia)	$\lambda, \$; \lambda \rightarrow$ ACEITA

Cada passo ou expande a variável do topo (por uma regra) ou consome um terminal casando-o com a fita. Quando o \$ aflora com a entrada esgotada, o AP aceita «0011». ✓

6.4 — Quais são as três formas de regra na construção $AP \rightarrow GLC$?

Enunciado.

Na conversão de um AP para GLC, listamos as regras usando variáveis A_{pq} . Quais são os três formatos?

Resolução.

Cada variável A_{pq} gera as cadeias que levam o AP do estado p ao estado q, começando e terminando com a pilha vazia. As regras cobrem os casos possíveis de como a pilha se comporta:

- $A_{pq} \rightarrow a A_{rs} b$ — quando o símbolo empilhado no primeiro movimento (lendo a) só é desempilhado no último (lendo b): os terminais das pontas ficam "casados", como um par de parênteses.
- $A_{pq} \rightarrow A_{pr} A_{rq}$ — quando a pilha esvazia num estado intermediário r, dividindo o trajeto em dois pedaços independentes.
- $A_{pp} \rightarrow \varepsilon$ — o caso base: um trajeto de tamanho zero, de um estado para si mesmo.

A variável inicial da gramática é $A_{(q_0, q_{aceita})}$. ✓

6.5 — Por que exigir que o AP esvazie a pilha antes de aceitar?

Enunciado.

Explique o papel do requisito "esvaziar a pilha antes de aceitar" na conversão AP → GLC.

Resolução.

A variável inicial $A_{(q_0, q_{aceita})}$ foi **definida** para gerar as cadeias que levam do estado inicial ao de aceitação **com a pilha vazia nas duas pontas**. Se o AP pudesse aceitar com a pilha ainda cheia, essas computações não seriam capturadas por essa variável, e a gramática geraria a linguagem errada.

Forçar o esvaziamento (um dos requisitos de padronização do AP) faz a condição "de pilha vazia a pilha vazia" da definição de A_{pq} coincidir exatamente com a condição de aceitação do AP — é o que garante $L(G) = L(AP)$. ✓

6.6 — Por que separar empilhar de desempilhar (não fazer os dois na mesma transição)?

Enunciado.

Explique por que a construção exige que cada transição só empilhe OU só desempilhe.

Resolução.

A construção das regras se baseia em rastrear a **altura da pilha** ao longo da computação — é isso que permite identificar o Caso 1 (um símbolo empilhado no início e retirado só no fim → terminais casados) e o Caso 2 (a pilha esvazia no meio → divide o trajeto).

Se uma transição fizesse empilhamento e desempilhamento ao mesmo tempo, não daria para acompanhar essa evolução de forma limpa. Por isso desdobramos cada transição mista «a, b ; c» em duas: uma que só desempilha (a, b ; λ) e outra que só empilha (λ , λ ; c), passando por um estado auxiliar. ✓

6.7 — Interprete a regra $A_{pq} \rightarrow a A_{rs} b$ como "parênteses"

Enunciado.

Explique a intuição por trás da regra $A_{pq} \rightarrow a A_{rs} b$.

Resolução.

Pense no símbolo empilhado (t) como um par de parênteses: o «a» inicial **abre** (empilha t) e o «b» final **fecha** (desempilha o mesmo t). Tudo o que acontece entre eles — a subcadeia gerada por A_{rs} — ocorre "dentro" desse par, num nível de pilha acima do t.

É a mesma estrutura recursiva das gramáticas casadas, como $S \rightarrow aSb$ (para $a^n b^n$) ou das expressões parentizadas: um delimitador de abertura casado com um de fechamento, com conteúdo aninhado no meio. Essa correspondência entre "empilhar/desempilhar" e "abrir/fechar" é justamente o que faz o AP e a GLC descreverem a mesma linguagem. ✓

